WareHouseApp – Inventory Management System

CS107.3 Object Oriented Programming with C# (Repeat) · Coursework report

1. Problem statement

Small and medium warehouses often track stock, customers and staff on paper or in scattered spreadsheets. This leads to stock-outs that are noticed too late, duplicated or inconsistent customer records, and no control over who may view or change sensitive data such as employee salaries.

WareHouseApp is a Windows Forms desktop application, built in C# on .NET Framework 4.7.2, that keeps inventory, customers and employees in one SQL Server database behind a login screen. Validation prevents bad data, low-stock alerts on the home dashboard show what needs reordering, and role-based access protects employee records. Together these give staff one reliable, searchable source of truth.

2. Requirements

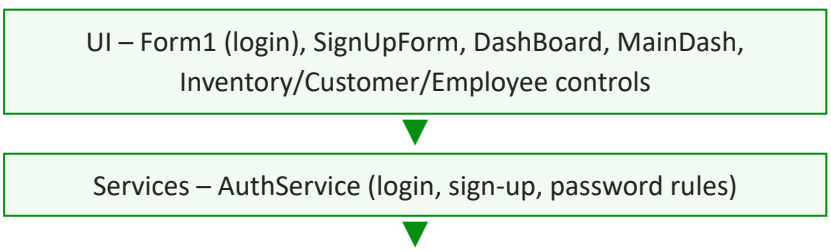
Target users: warehouse administrators (full access) and shipping operators (day-to-day stock and customer work, no access to employee records).

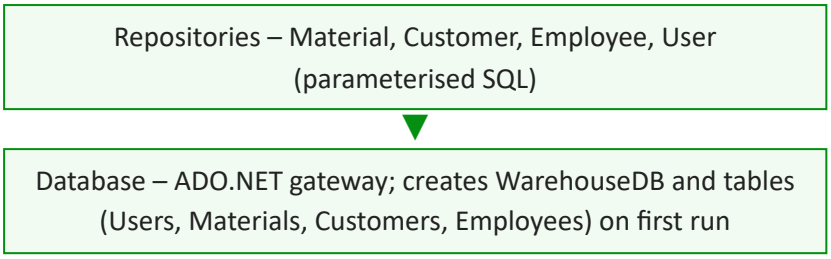
Feature	Why it is included
Sign up & login	Only authorised staff may use the system; passwords are stored as salted PBKDF2 hashes, never in plain text. A default admin is created on first run.
Inventory management	Add, edit, delete and search items with quantity, price, category, location and reorder level; low stock is flagged on the dashboard.
Customer management	Maintain customer contact and company details with phone/email validation.
Employee management	Maintain staff records (NIC, position, salary, hire date); restricted to Admin.

Non-functional: every feature is persisted in SQL Server; all user input is validated; all failures are caught and shown as friendly messages; the UI is interactive via mouse and keyboard (Enter submits the login form).

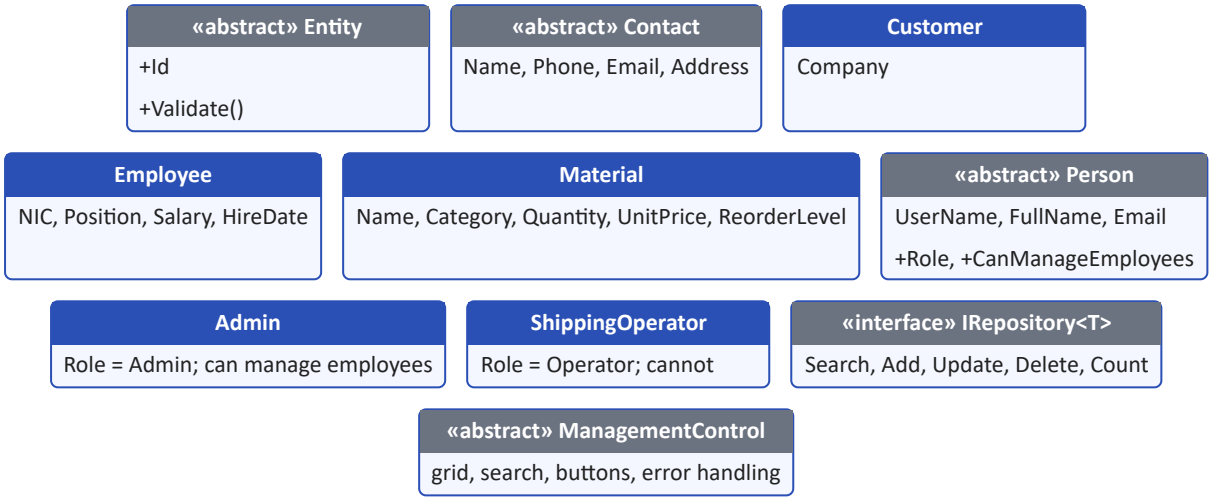
3. Design

Architecture. A layered design keeps the user interface independent of data access:





Class structure.



Inheritance: Contact and Material and Person derive from Entity; Customer and Employee derive from Contact; Admin and ShippingOperator derive from Person. Material, Customer and Employee repositories implement IRepository<T>. InventoryControl, CustomerControl and EmployeeControl derive from ManagementControl.

Process interaction. Program initialises the database, then loops Login → Dashboard. A screen builds an entity from the form fields; the entity validates itself; the repository runs a parameterised SQL command through Database; any exception is translated and displayed by ManagementControl.

Why appropriate. The template-method base class removed duplicated screen code, so a new module needs only its fields and queries. Polymorphism (Role, CanManageEmployees, IRepository) keeps role logic and data access swappable, and parameterised queries prevent SQL injection. Custom exceptions (ValidationException, AuthenticationException, DataAccessException) separate user mistakes from system faults.

Version control. The project is managed in Git with branches *master*, *Designing* (UI design work, merged through a pull request) and a working branch. Commit history shows the progression: project files added, main dashboard built, start position/login layout, database integration, followed by commits for sign-up/login, inventory, customer and employee modules. Committing one feature at a time kept changes small and reversible. The database is created from code on first run (Database.Initialize), so its schema is version-controlled with the source.

4. Screenshots

[Insert screenshot: Login and Sign Up]

[Insert screenshot: Dashboard with statistics]

[Insert screenshot: Inventory, Customer and Employee screens]

[Insert screenshot: SQL Server tables / database snapshot]

5. Personal reflection

The abstract ManagementControl base class and the repository interface worked well: they kept three screens consistent and made adding validation and error handling a one-place job. Letting the app create its own database removed setup steps and the dependency on an attached .mdf file. Things that went less well were the starter project's missing resource files, which stopped it compiling, and the hard-coded login, which had to be replaced. Hand-coding some layouts instead of using the designer was quicker to write but harder to preview. In the next project I will write unit tests for the validation rules earlier, commit smaller changes more often, and agree the database schema before building the screens.

6. Future work

The application could be extended with stock movement history (goods in/out), customer orders and shipping status, printable and exportable reports, barcode scanning, a change-password and user-management screen, and a language option (the starter already contains a Language button). For sustainability I would move the connection string to a configuration file, add a data-access layer interface so the database can be swapped or mocked, add automated tests and logging, back up the database on a schedule, and publish an installer so updates are easy to deploy.